COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	12/09/2026	**DATE SUBMITTED:**	16/09/2026
NAME:	Seth Carlo T. Miraflor	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 8.1 Sorting Algorithms Pt.2

1. PROCEDURE OUTPUT

What procedures did you perform?

sorting.h | main.cpp

HOA 8.1 > ILO A > main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

using namespace std;

const int SIZE = 100;


// Function to display array
void displayArray(int arr[], int size)
{
    for (int i = 0; i < size; i++)
    {
        cout << arr[i] << " ";

        if ((i + 1) % 10 == 0)
        {
            cout << endl;
        }
    }
}


// Function to copy array
void copyArray(int original[], int copy[], int size)
{
    for (int i = 0; i < size; i++)
    {
        copy[i] = original[i];
    }
}


int main()
```

sorting.h | main.cpp

HOA 8.1 > ILO A > main.cpp > main()

```cpp
int main()
{
    int original[SIZE];
    int shell[SIZE];
    int merge[SIZE];
    int quick[SIZE];

    // Random number generator
    srand(time(0));

    // Create 100 random values
    for (int i = 0; i < SIZE; i++)
    {
        original[i] = rand() % 1000 + 1;
    }

    // Copy the original array
    copyArray(original, shell, SIZE);
    copyArray(original, merge, SIZE);
    copyArray(original, quick, SIZE);

    cout << "Original Array:" << endl;
    displayArray(original, SIZE);

    cout << endl;

    // Shell Sort
    shellSort(shell, SIZE);

    cout << "After Shell Sort:" << endl;
    displayArray(shell, SIZE);

    cout << endl;

    // Merge Sort
    mergeSort(merge, 0, SIZE - 1);
```

```
72
73      cout << "After Merge Sort:" << endl;
74      displayArray(merge, SIZE);
75
76      cout << endl;
77
78      // Quick Sort
79      quickSort(quick, 0, SIZE - 1);
80
81      cout << "After Quick Sort:" << endl;
82      displayArray(quick, SIZE);
83
84      return 0;
85  }
```

sorting.h main.cpp

HOA 8.1 > ILO A > sorting.h > quickSort(int [], int, int)

```cpp
#ifndef SORTING_H
#define SORTING_H

// Shell Sort
void shellSort(int arr[], int size)
{
    for (int interval = size / 2; interval > 0; interval = interval / 2)
    {
        for (int i = interval; i < size; i++)
        {
            int temp = arr[i];
            int j = i;

            while (j >= interval && arr[j - interval] > temp)
            {
                arr[j] = arr[j - interval];
                j = j - interval;
            }

            arr[j] = temp;
        }
    }
}


// Merge function
void merge(int arr[], int left, int middle, int right)
{
    int n1 = middle - left + 1;
    int n2 = right - middle;

    int leftArray[n1];
    int rightArray[n2];
```

```
34
35      for (int i = 0; i < n1; i++)
36      {
37          leftArray[i] = arr[left + i];
38      }
39
40      for (int j = 0; j < n2; j++)
41      {
42          rightArray[j] = arr[middle + 1 + j];
43      }
44
45      int i = 0;
46      int j = 0;
47      int k = left;
48
49      while (i < n1 && j < n2)
50      {
51          if (leftArray[i] <= rightArray[j])
52          {
53              arr[k] = leftArray[i];
54              i++;
55          }
56          else
57          {
58              arr[k] = rightArray[j];
59              j++;
60          }
61
62          k++;
63      }
64
```

```
64
65      while (i < n1)
66      {
67          arr[k] = leftArray[i];
68          i++;
69          k++;
70      }
71
72      while (j < n2)
73      {
74          arr[k] = rightArray[j];
75          j++;
76          k++;
77      }
78  }
79
80
81  // Merge Sort
82  void mergeSort(int arr[], int left, int right)
83  {
84      if (left < right)
85      {
86          int middle = (left + right) / 2;
87
88          mergeSort(arr, left, middle);
89          mergeSort(arr, middle + 1, right);
90
91          merge(arr, left, middle, right);
92      }
93  }
94
95
96  // Partition for Quick Sort
97  int partition(int arr[], int low, int high)
```

```
96  // Partition for Quick Sort
97  int partition(int arr[], int low, int high)
98  {
99      int pivot = arr[high];
100
101     int i = low - 1;
102
103     for (int j = low; j < high; j++)
104     {
105         if (arr[j] < pivot)
106         {
107             i++;
108
109             int temp = arr[i];
110             arr[i] = arr[j];
111             arr[j] = temp;
112         }
113     }
114
115     int temp = arr[i + 1];
116     arr[i + 1] = arr[high];
117     arr[high] = temp;
118
119     return i + 1;
120 }
```

```
121
122
123   // Quick Sort
124   void quickSort(int arr[], int low, int high)
125   {
126       if (low < high)
127       {
128           int pivot = partition(arr, low, high);
129
130           quickSort(arr, low, pivot - 1);
131           quickSort(arr, pivot + 1, high);
132       }
133   }
134
135   #endif
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.
ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

- Yes, we can use other sorting algorithms to sort the left and right sublists in quick sort for example, we can
- use insertion sort after doing the partition

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- For me, both are fast sorting algorithms because they can handle many elements efficiently

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.